

American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Final-Term Project Report

Summer 2025-2026

Section: D

Group: 09

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Eva Jahan ID: 23-55536-3	Data Preprocessing (tokenization, case folding, lemmatization, stop-word and punctuation removal); BERT fine-tuning experiments and results for Dataset 1 and Dataset 2; **Report Assembly**
		RoBERTa fine-tuning experiments and results table for Dataset 1 and Dataset 2; results visualization; final Results comparison.
2.	Name: Hamed Mohammed Adel ID: 23-55566-3	Stacked Ensemble (BERT + RoBERTa) implementation and evaluation; validation/test split creation and experiment logging.
		Dataset loading and label encoding for both datasets; model checkpoint management and evaluation metric computation (accuracy, precision, recall, F1).

Authorship Attribution Using Classical NLP, Transformer-Based Models, and Ensemble Learning

Eva Jahan, Hamed Mohammed Adel, American International University-Bangladesh
Dhaka, Bangladesh
{23-55536-3, 23-55566-3}@student.aiub.edu

1 Introduction

This report presents the extension of authorship-attribution project previously built on a classical NLP pipeline(tokenization, case folding, lemmatization, stop-word and punctuation removal, Bag-of-Words and TF-IDF vectorization) and trained Naïve Bayes, Logistic Regression, and Support Vector Machine classifiers on two datasets: Dataset 1, the Reuters 50-50 (C50) new corpus with 50 candidate authors; and Dataset 2, the Spooky Author Identification corpus with 3 candidate authors Edger Allan Poe(EAP), H.P. Lovecarft(HPL), and Mary Shelley(MWS).

Each dataset was split 70% train, 15% validation; 15% test with stratification on the author lab.

Two transformer-based language models were fine-tuned for classification: bert-base-uncased and roberta-base, each trained for up to 5 epochs with max sequence length of 128 tokens, early stopping, Warmup Ratio of 0.1, Dropout of 0.1 and with AdamW Optimizer across an 8-configuration paramerter grid formed from learning rate {0.00002, 0.00003}, batch size {16,32}, and weight decay {0.01,0.1}. The validation-set softmax probabilities from the best BERT and best RoBERTa configuration for each dataset were then combined into a stacked ensemble, with a Logistic Regression meta-learner trained on the concatenated probability vectors.

This report analyses the classical NLP results, BERT and RoBERTa parameter search, discusses the stacked ensemble, compares all methods and reflects on the limitations of the overall approach.

2 Midterm Result Analysis

In Midterm compared three classical classifiers (Multinomial Naïve Bayes, Logistic Regression, and Linear SVM) against two feature representations (Bag-of-Words and TF-IDF) on both datasets, using the tokenization, case folding. Lemmatization, stop-word and punctuation removal text preprocessing pipeline. The full results of the mid-term report, are shown below

Algorithm	Repr.	Dataset 1				Dataset 2			
		Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Naïve Bayes	BoW	0.6544	0.6814	0.6544	0.6502	0.8368	0.8383	0.8368	0.8370
	TF-IDF	0.6320	0.6496	0.6320	0.6198	0.8207	0.8291	0.8207	0.8200
Logistic Regression	BoW	0.6740	0.6850	0.6740	0.6720	0.8110	0.8140	0.8110	0.8107
	TF-IDF	0.6424	0.6558	0.6424	0.638	0.8169	0.8199	0.8169	0.8165
SMV	BoW	0.6696	0.6818	0.6696	0.6669	0.7985	0.8000	0.7982	0.7982
	TF-IDF	0.6692	0.6758	0.6692	0.6649	0.8192	0.8204	0.8192	0.8191

Overall, accuracy is consistently higher on the Spooky Author dataset than on Reuters 50-50, which is expected since Spooky is a 3-class problem while Reuters is a much harder 50-class problem. On Reuters, Logistic Regression with BoW gives the best result (67.4%); on Spooky, Support Vector Machine with TF-IDF gives the best result (83.5%). Across both datasets, TF-IDF does not uniformly beat BoW - the better representation depends on the algorithm, which is why both were required and compared for every model.

3 Selecting the Best BERT model

BERT (bert-base-uncased) was selected as the primary transformer model because it is a bidirectional, pre-trained encoder that builds contextualized word representations rather than treating text as an unordered bag of counts. Unlike Bag-of-Words or TF-IDF, BERT can capture word order, syntactic patterns, and long-range dependencies, all of which can carry an author's stylistic signature (sentence structure, function-word usage, phrasing) beyond simple vocabulary frequency. It is also a natural, widely-used first choice for a sequence-classification fine-tuning task because of its strong general-purpose language understanding and manageable size for fine-tuning on a single GPU.

An 8-configuration grid (learning rate, batch size, weight decay) was fine-tuned on both datasets for up to 5 epochs. Results:

Model	Learning Rate	Batch Size	Weight Decay	Dataset 1(Reuters 50-50)				Dataset 2(Spooky Author)			
				Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
BERT	0.00002	16	0.01	0.6333	0.6384	0.6333	0.6075	0.8014	0.8041	0.8014	0.8016
	0.00003	16	0.01	0.6813	0.6999	0.6813	0.6599	0.8069	0.8084	0.8069	0.8069
	0.00002	32	0.01	0.5907	0.5992	0.5907	0.5520	0.7963	0.7981	0.7963	0.7963
	0.00003	32	0.01	0.6613	0.6690	0.6613	0.6312	0.8106	0.8112	0.8106	0.8106
	0.00002	16	0.1	0.6560	0.6555	0.6560	0.6225	0.8025	0.8053	0.8025	0.8027
	0.00003	16	0.1	0.6720	0.6858	0.6720	0.6516	0.8103	0.8116	0.8103	0.8104
	0.00002	32	0.1	0.5960	0.5809	0.5960	0.5558	0.7963	0.7979	0.7963	0.7963
	0.00003	32	0.1	0.6520	0.6797	0.6520	0.6222	0.8110	0.8115	0.8110	0.8110

On Dataset 1, Experiment 2 where learning rate = 0.00003, batch size = 16, weight decay = 0.01 gave the best BERT configuration with validation accuracy 68.13%, Precision 69.99%, Recall 68.13% and weighted F1 is 0.6599. Smaller batch size (16) consistently outperformed batch size 32 across the gird, suggesting the model benefits from more frequent gradient updates on this relatively small, 50-class training set.

On Dataset 2, Experiment 8 where learning rate = 0.00003, batch size = 32, weight decay = 0.1 gave the best BERT configuration with validation accuracy 81.10 %, Precision 81.15%, Recall 81.10% and weighted F1 is 0.8110.

These two configurations (Experiment 2 for dataset 1, Experiment 8 for dataset 2) were carried forward as the “best BERT” model for the ensemble in stack ensemble meta-learner.

4 Selecting the Best BERT variant model

RoBERTa (roberta-base) was chosen as the BERT variant because it uses the same transformer-encoder architecture as BERT but changes the pre-training technic it drops the Next-Sentence-Prediction objective, trains with dynamic masking, uses much larger batches and about 10 times more pre-training text, and tokenizes with a byte-level BPE vocabulary instead of BERT's WordPiece vocabulary. These changes typically give RoBERTa stronger, better-calibrated contextual representations than BERT, making it a natural second model to compare against and to combine in an ensemble.

The identical 8-configuration parameter grid used for BERT was fine-tuned for RoBERTa on both datasets, for up to 5 epochs. Results:

Model	Learning Rate	Batch Size	Weight Decay	Dataset 1(Reuters 50-50)				Dataset 2(Spooky Author)			
				Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
RoBERTa	0.00002	16	0.01	0.6867	0.6871	0.6867	0.6733	0.7851	0.7865	0.7851	0.7851
	0.00003	16	0.01	0.6973	0.6959	0.6973	0.6872	0.7902	0.7912	0.7902	0.7901
	0.00002	32	0.01	0.6387	0.6354	0.6387	0.6154	0.7827	0.7850	0.7827	0.7829
	0.00003	32	0.01	0.6720	0.6948	0.6720	0.6587	0.7885	0.7908	0.7885	0.7886
	0.00002	16	0.1	0.6867	0.6871	0.6867	0.6733	0.7847	0.7862	0.7847	0.7848
	0.00003	16	0.1	0.6973	0.6961	0.6973	0.6873	0.7895	0.7905	0.7895	0.7894
	0.00002	32	0.1	0.6387	0.6354	0.6387	0.6154	0.7824	0.7846	0.7824	0.7825
	0.00003	32	0.1	0.6720	0.6923	0.6720	0.6579	0.7888	0.7911	0.7888	0.7889

On Dataset 1, Experiment 6 where learning rate = 0.00003, batch size = 16, weight decay = 0.1 gave the best RoBERTa configuration with validation accuracy 69.73%, Precision 69.61%, Recall 69.73% and weighted F1 is 0.6873. this is RoBERTa's single best result across both models on Dataset 1, exceeding the best BERT configuration (68.13%) by about 1.6 percentage points. As with BERT, batch size 16 clearly outperformed batch size 32 throughout the grid.

On Dataset 2, Experiment 2 (learning rate 3e-5, batch size 16, weight decay 0.01) gave the best RoBERTa configuration, with validation accuracy 79.02%, Precision 78.12%, and weighted F1 0.7901. This is roughly 2 percentage points below the best BERT result on the same dataset (81.10%), the one case in this project where RoBERTa did not outperform BERT - most likely because the text seen by both models had already been lowercased, lemmatized, and stripped of punctuation and stop words during mid-term preprocessing, which removes exactly the kind of casing and punctuation cues that RoBERTa's byte-level tokenizer is designed to exploit. Experiment 6 (Dataset 1) and Experiment 2 (Dataset 2) were carried forward as the "best RoBERTa" model for the ensemble in Section 5.

5 Ensemble Mode

The ensemble is a stacking model that combines the best BERT configuration with the best RoBERTa configuration for each dataset. Each fine tuned model outputs a softmax probability distribution over the candidate authors the validation-set probability vectors from BERT and RoBERTa are concatenated into a single meta-feature vector, and a Logistic Regression meta-learner (max_iter=1000) is trained on these meta-features against the true validation labels. At test time, the same concatenation is applied to the BERT and RoBERTa test-set probabilities and passed through the trained meta-learner to obtain the final ensemble prediction. This lets the ensemble learn, per class, how much to trust each base model rather than averaging their outputs with fixed, equal weight.

Model	Dataset 1				Dataset 2			
	Acc	**Prec**	**Rec**	**F1**	**Acc**	**Prec**	**Rec**	**F1**
Ensemble	0.6933	0.6980	0.6933	0.6755	0.8352	0.8358	0.8352	0.8352

On Dataset 2, the ensemble clearly improves over both base models: 83.52% accuracy versus 81.10% for BERT alone and 79.02% for RoBERTa alone, showing the meta-learner successfully exploits the complementary information in the two models' probability outputs.
On Dataset 1, the ensemble reaches 69.33% accuracy, a clear improvement over BERT alone (68.13%), but it falls just short of RoBERTa alone (69.73%), the single best result on this dataset. This shows that stacking does not guarantee an improvement over the stronger of the two base models.

6 Overall Result Analysis

The table below summarizes the single best-performing configuration of every method covered in this project:

Model	Dataset 1				Dataset 2			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Naïve Bayes	0.6544	0.6814	0.6544	0.6502	0.8368	0.8383	0.8368	0.8370
Logistic Regression	0.6740	0.6850	0.6740	0.6720	0.8169	0.8199	0.8169	0.8165
Support Vector Machine	0.6696	0.6818	0.6696	0.6669	0.8192	0.8204	0.8192	0.8191

BERT	0.6813	0.6999	0.6813	0.6599	0.8110	0.8115	0.8110	0.8110
RoBERTa	0.6973	0.6959	0.6973	0.6872	0.7902	0.7912	0.7902	0.7901
ENSEMBLE	0.6933	0.6980	0.6933	0.6755	0.8352	0.8358	0.8352	0.8352

On Dataset 1, every transformer-based method clearly beats the best classical mid-term model RoBERTa alone (69.73%) is the strongest single result, followed by the Ensemble (69.33%) and BERT alone (68.13%), all ahead of the best mid-term Logistic Regression + BoW result (67.40%). This confirms that pre-trained contextual language models bring a real advantage when the classification problem is fine-grained (50 classes) and the labeled data per class is comparatively scarce (only around 70 documents per author in the training split): the sparse Bag-of-Words/TF-IDF features from the classical pipeline are simply not expressive enough to reliably tell apart 50 writing styles, whereas BERT and RoBERTa arrive with rich, pre-trained language understanding that partially compensates for the limited per-class data.

On Dataset 2, the easier 3-way task with roughly four times more training data than Dataset 1, the picture reverses: the best classical model (Naive Bayes + BoW, 83.68%) is essentially tied with, and marginally ahead of, the Ensemble (83.52%), and both comfortably beat BERT alone (81.10%) and RoBERTa alone (79.02%). Two factors likely explain this. First, with only 3 classes and thousands of training examples per class, a simple linear/probabilistic bag-of-words classifier can already come close to saturating the separable signal in the vocabulary, leaving little headroom for a more expressive model to add value. Second, the text fed to every model classical and transformer alike - had already been lowercased, lemmatized, and stripped of stop words and punctuation during the shared mid-term preprocessing step; this destroys exactly the stylistic signals (capitalization habits, punctuation choices, function-word order) that transformer tokenizers are best placed to exploit, effectively handicapping BERT and RoBERTa relative to how they would perform on raw, unprocessed text.

Overall, the results suggest that transformer fine-tuning and ensembling are most valuable on the harder, more fine-grained classification problem (Dataset 1), while for an easier problem with abundant data and heavily normalized text (Dataset 2), a well-tuned classical Bag-of-Words model remains a very strong, compute-cheap baseline that transformer models fine-tuned on the same normalized text do not clearly surpass.

Metric Curve:

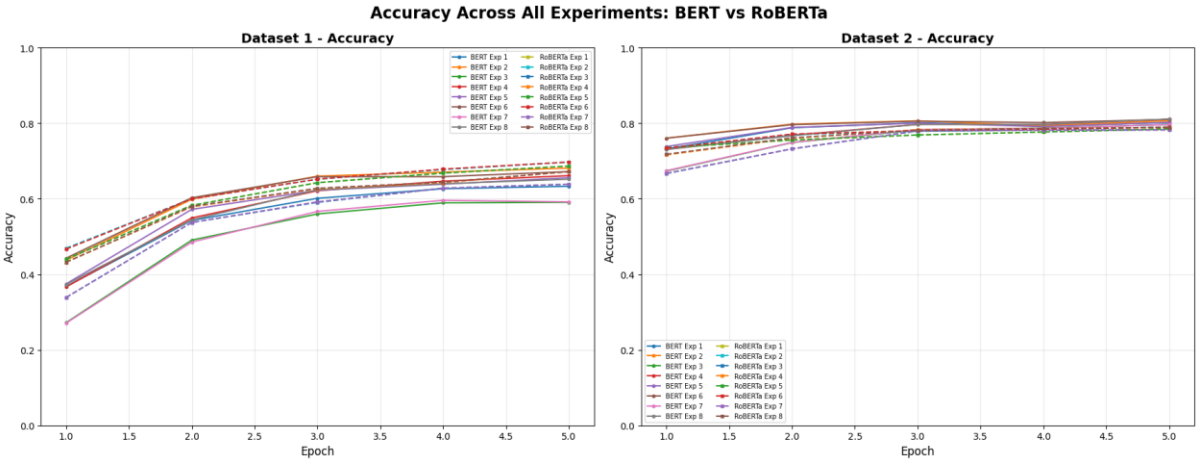


Fig.1: Dataset 1&2 all experiment Accuracy curves

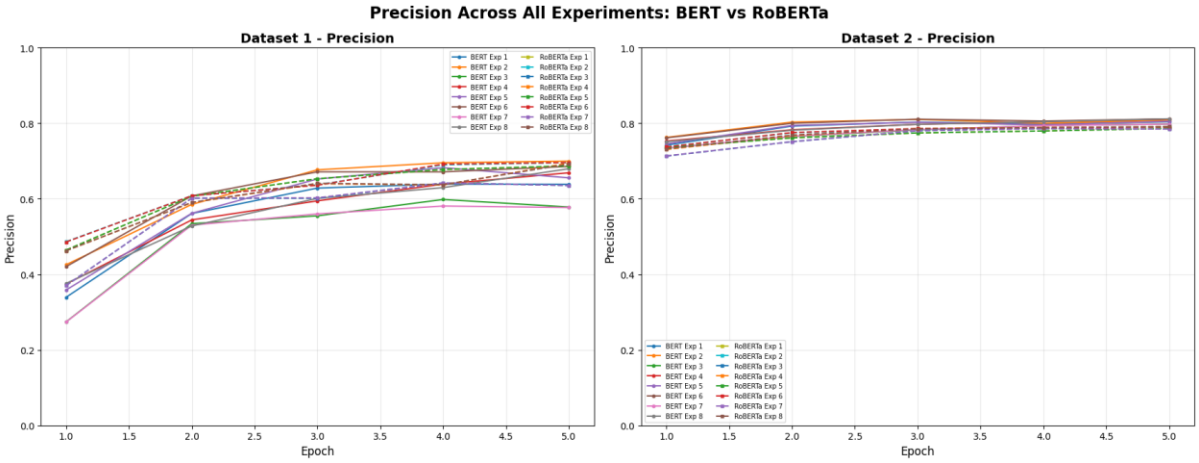


Fig.2: Dataset 1&2 all experiment Precision curves

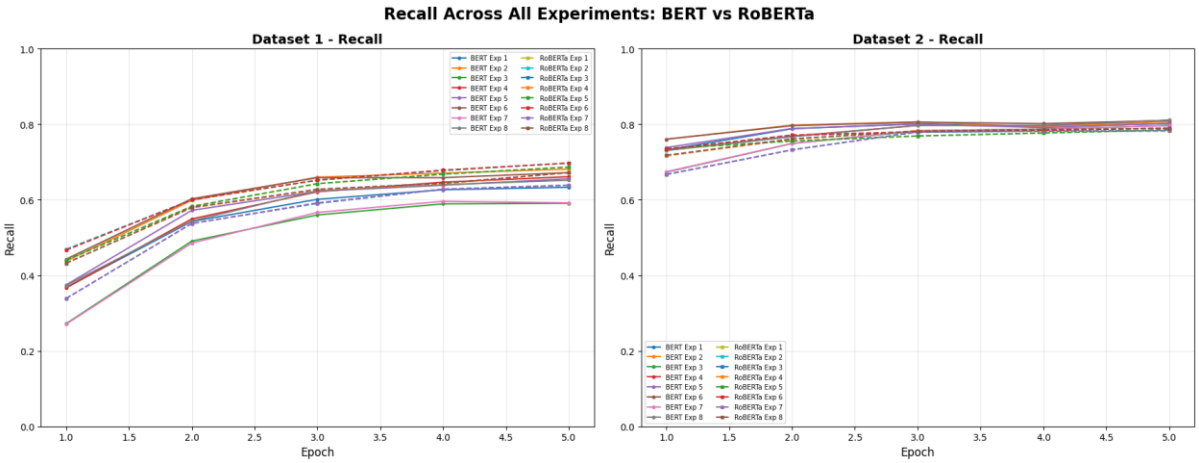


Fig.3: Dataset 1&2 all experiment Recall curves

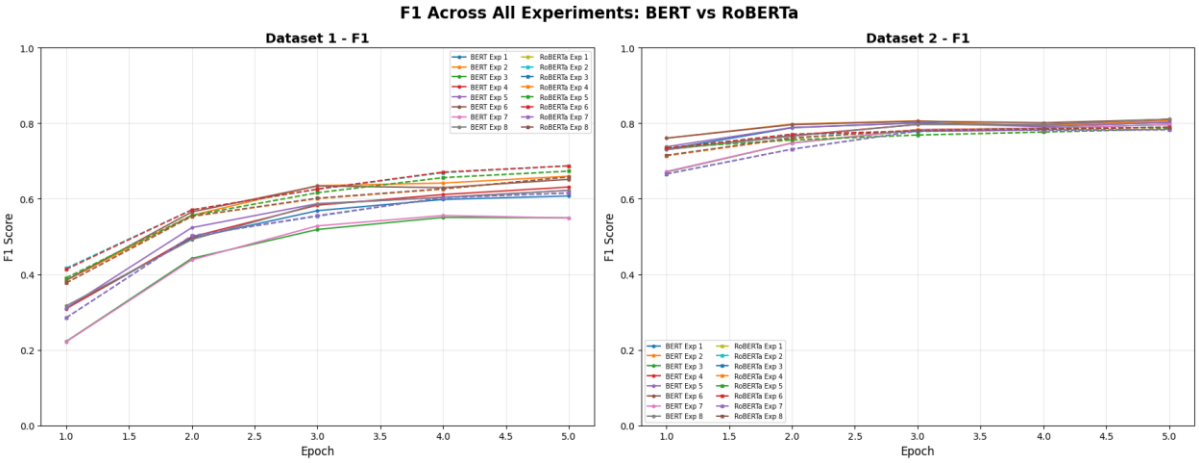


Fig.4: Dataset 1&2 all experiment F1 curves

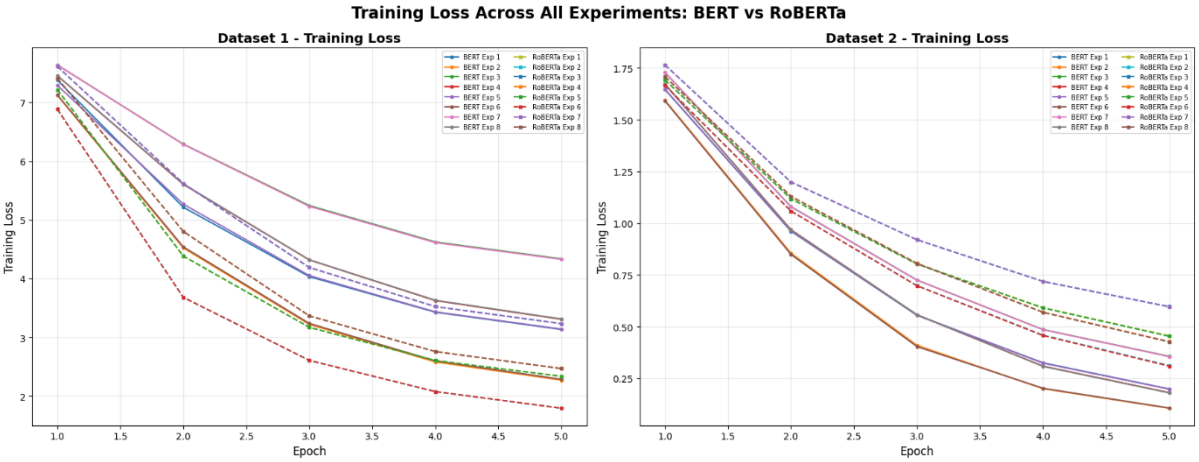


Fig.5: Dataset 1&2 all experiment Training Loss curves

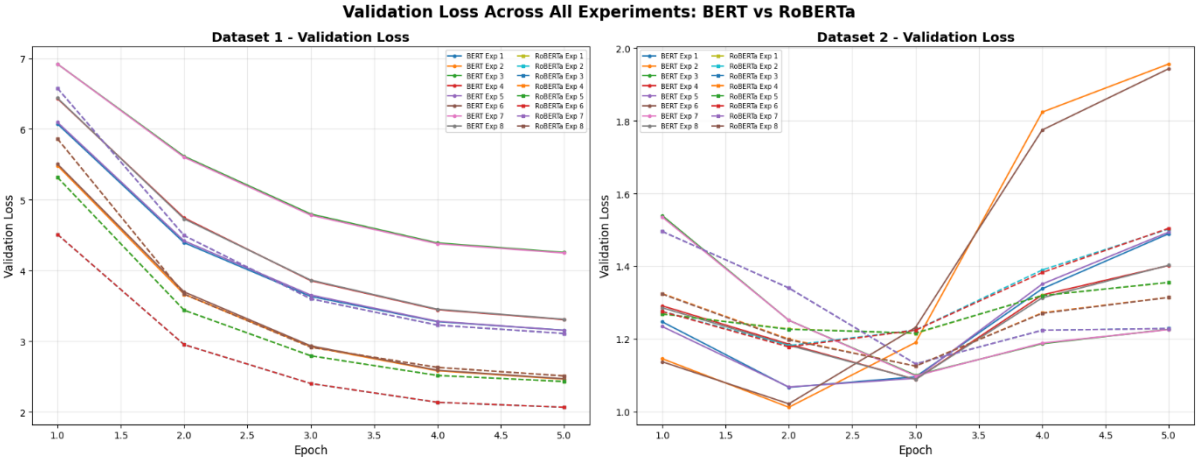


Fig.6: Dataset 1&2 all experiment Validation Loss curves

7 Limitations

The project has several limitations that should be kept in mind when interpreting the results:

1. Preprocessing mismatch for transformers: the same lowercased, lemmatized, stop-word- and punctuation-stripped text used for the mid-term classical models was also fed to BERT and RoBERTa. Transformer tokenizers are designed to exploit case, punctuation, and natural word order, so this normalization likely understated how well BERT/RoBERTa could perform on raw text.
2. Limited parameter search: only 8 grid combinations (learning rate, batch size, weight decay) were tried per model; learning-rate schedules, number of epochs, dropout, and larger model variants (bert-large, roberta-large) were not explored due to compute and time constraints.
3. Single train/validation/test split: results are based on one stratified 70/15/15 split rather than k-fold cross-validation, so reported metrics - especially for Dataset 1, where each of the 50 classes has only about 15 validation and 15 test examples - carry non-trivial sampling variance.
4. Simple ensemble: the stacked ensemble uses only two base models and a plain Logistic Regression meta-learner; more sophisticated stacking (additional base learners, tuned/weighted averaging, calibration of base-model probabilities) was not explored and might close the gap seen where the ensemble slightly underperformed the best single model on Dataset 1.
5. Truncated sequence length: inputs were truncated to 128 tokens, which may drop author-specific signal from longer Reuters news articles.
6. Compute-constrained training: training was run on a single Kaggle T4 GPU for up to 5 epochs with early stopping (patience 2), which may under-train some configurations relative to what longer training or more powerful hardware would allow.

8 Conclusion

This project extended a classical NLP authorship-attribution pipeline into a study of transformer fine-tuning and ensembling. Two transformer encoders, BERT and RoBERTa, were each fine-tuned across an 8-configuration parameters on two datasets of very different difficulty (50-way Reuters authorship versus 3-way Spooky Author identification), and their best configurations were combined into a stacked ensemble with a Logistic Regression meta-learner.

The key finding is that no single method dominates across both settings. On the harder, fine-grained 50-author task, transformer fine-tuning provides a clear, consistent advantage over classical Bag-of-Words/TF-IDF models, with RoBERTa (69.73% accuracy) and the ensemble (69.33%) both outperforming the mid-term baseline (67.40%). On the easier 3-author task with abundant training data, a simple, inexpensive classical model (Naive Bayes with Bag-of-Words, 83.68%) remained competitive with, and marginally better than, the transformer ensemble (83.52%). This is a useful, practically-minded takeaway: the added complexity and compute cost of fine-tuning large pre-trained language models pays off most clearly when the classification problem is genuinely hard and data-scarce per class, and is not automatically justified for easier, well-resourced problems.

Project Code

Data Preprocessing

```
def clean_dataset(df):

    df = df.copy()

    df = df.dropna(subset=["processed_text", "author"])

    df["processed_text"] = df["processed_text"].astype(str).str.strip()

    df = df[df["processed_text"] != ""]

    df = df.reset_index(drop=True)

    return df


def encode_labels(df):

    df = df.copy()

    label_encoder = LabelEncoder()

    df["label"] = label_encoder.fit_transform(df["author"])

    return df, label_encoder


def create_splits(df):

    train_df, temp_df = train_test_split(
        df, test_size=0.30, random_state=SEED, stratify=df["label"]
    )

    val_df, test_df = train_test_split(
        temp_df, test_size=0.50, random_state=SEED, stratify=temp_df["label"]
    )

    return (
        train_df.reset_index(drop=True),
        val_df.reset_index(drop=True),
        test_df.reset_index(drop=True),
    )



def create_hf_dataset(df):

    dataset = Dataset.from_pandas(df[["processed_text", "label"]], preserve_index=False)

    return dataset


def tokenize_function(examples):

    return tokenizer(examples["processed_text"], truncation=True, max_length=MAX_LENGTH)
```

BERT Code (Best-Performing Configuration)

```
experiment_configs = [

    {"learning_rate": 2e-5, "batch_size": 16, "weight_decay": 0.01},

    {"learning_rate": 3e-5, "batch_size": 16, "weight_decay": 0.01},

    {"learning_rate": 2e-5, "batch_size": 32, "weight_decay": 0.01},

    {"learning_rate": 3e-5, "batch_size": 32, "weight_decay": 0.01},

    {"learning_rate": 2e-5, "batch_size": 16, "weight_decay": 0.1},

    {"learning_rate": 3e-5, "batch_size": 16, "weight_decay": 0.1},

    {"learning_rate": 2e-5, "batch_size": 32, "weight_decay": 0.1},

    {"learning_rate": 3e-5, "batch_size": 32, "weight_decay": 0.1},
]
WORKING_DIR = "/kaggle/working"

MODEL_NAME = "bert-base-uncased"

MAX_LENGTH = 128

EPOCHS = 5

WARMUP_RATIO = 0.1

DROPOUT = 0.1

EARLY_STOPPING_PATIENCE = 2



def train_one_experiment(
    train_dataset,
    val_dataset,
    test_dataset,
    num_labels,
    config,
    dataset_name,
    experiment_number,
):

    print("\n" + "=" * 70)

    print(f"DATASET: {dataset_name}")
    print(f"EXPERIMENT: {experiment_number}")

    print(f"Learning Rate: " f"{config['learning_rate']}")

    print(f"Batch Size: " f"{config['batch_size']}")

    print(f"Weight Decay: " f"{config['weight_decay']}")

    print("=" * 70)

    model = AutoModelForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=num_labels,
        hidden_dropout_prob=DROPOUT,
        attention_probs_dropout_prob=DROPOUT,
    )
```

```
output_dir = os.path.join(
    WORKING_DIR, f"bert_{dataset_name}_exp_{experiment_number}"
)


training_args = TrainingArguments(
    output_dir=output_dir,
    learning_rate=config["learning_rate"],
    per_device_train_batch_size=config["batch_size"],
    per_device_eval_batch_size=config["batch_size"],
    num_train_epochs=EPOCHS,
    weight_decay=config["weight_decay"],
    warmup_ratio=WARMUP_RATIO,
    optim="adamw_torch",
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,
    save_total_limit=1,
    report_to="none",
    fp16=torch.cuda.is_available(),
    seed=SEED,
)



trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    callbacks=[
        EarlyStoppingCallback(early_stopping_patience=EARLY_STOPPING_PATIENCE)
    ],
)


trainer.train()


best_checkpoint = trainer.state.best_model_checkpoint

best_metric = trainer.state.best_metric

print("\nBEST CHECKPOINT:")
print(best_checkpoint)

print("\nBEST VALIDATION F1:")
print(best_metric)

validation_results = trainer.evaluate()

val_prediction_output = trainer.predict(val_dataset)

val_logits = val_prediction_output.predictions

val_probabilities = torch.softmax(torch.tensor(val_logits), dim=1).cpu().numpy()

val_labels = val_prediction_output.label_ids


test_prediction_output = trainer.predict(test_dataset)
```

```
test_logits = test_prediction_output.predictions

test_probabilities = torch.softmax(torch.tensor(test_logits), dim=1).cpu().numpy()

test_labels = test_prediction_output.label_ids


prediction_dir = os.path.join(WORKING_DIR, "bert_predictions")

os.makedirs(prediction_dir, exist_ok=True)

np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_val_prob.npy"
    ),
    val_probabilities,
)

np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_val_labels.npy"
    ),
    val_labels,
)

np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_test_prob.npy"
    ),
    test_probabilities,
)

np.save(
    os.path.join(
        prediction_dir,
        f"{dataset_name}_exp_" f"{experiment_number}_test_labels.npy",
    ),
    test_labels,
)

result = {
    "experiment": experiment_number,
    "learning_rate": config["learning_rate"],
    "batch_size": config["batch_size"],
    "weight_decay": config["weight_decay"],
    "best_validation_f1": best_metric,
    "accuracy": validation_results["eval_accuracy"],
    "precision": validation_results["eval_precision"],
    "recall": validation_results["eval_recall"],
    "f1": validation_results["eval_f1"],
}

print("\nVALIDATION RESULT:")

print(result)

return result
```

RoBERTa Code (Best-Performing Configuration)

```
def roberta_train_one_experiment(
    train_dataset,
    val_dataset,
    test_dataset,
    num_labels,
    config,
    dataset_name,
    experiment_number,
):
    print("\n" + "=" * 70)
    print(f"DATASET: {dataset_name}")
    print(f"EXPERIMENT: {experiment_number}")

    print(f"Learning Rate: " f"{config['learning_rate']}")
    print(f"Batch Size: " f"{config['batch_size']}")
    print(f"Weight Decay: " f"{config['weight_decay']}")
    print("=" * 70)
    model = AutoModelForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=num_labels,
        hidden_dropout_prob=DROPOUT,
        attention_probs_dropout_prob=DROPOUT,
    )


    output_dir = os.path.join(
        WORKING_DIR, f"roberta_{dataset_name}_exp_{experiment_number}"
    )
    training_args = TrainingArguments(
        output_dir=output_dir,
        learning_rate=config["learning_rate"],
        per_device_train_batch_size=config["batch_size"],
        per_device_eval_batch_size=config["batch_size"],
        num_train_epochs=EPOCHS,
        weight_decay=config["weight_decay"],
        warmup_ratio=WARMUP_RATIO,
        optim="adamw_torch",
        eval_strategy="epoch",
        save_strategy="epoch",
        logging_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="f1",
        greater_is_better=True,
        save_total_limit=1,
        report_to="none",
        fp16=torch.cuda.is_available(),
        seed=SEED,
    )
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        data_collator=data_collator,
        compute_metrics=compute_metrics,
        callbacks=[
            EarlyStoppingCallback(early_stopping_patience=EARLY_STOPPING_PATIENCE)
        ],
    )


    trainer.train()
    best_checkpoint = trainer.state.best_model_checkpoint
    best_metric = trainer.state.best_metric
```

```
print("\nBEST CHECKPOINT:")
print(best_checkpoint)
print("\nBEST VALIDATION F1:")
print(best_metric)
validation_results = trainer.evaluate()
val_prediction_output = trainer.predict(val_dataset)
val_logits = val_prediction_output.predictions
val_probabilities = torch.softmax(torch.tensor(val_logits), dim=1).cpu().numpy()
val_labels = val_prediction_output.label_ids
test_prediction_output = trainer.predict(test_dataset)
test_logits = test_prediction_output.predictions
test_probabilities = torch.softmax(torch.tensor(test_logits), dim=1).cpu().numpy()
test_labels = test_prediction_output.label_ids
prediction_dir = os.path.join(WORKING_DIR, "roberta_predictions")
os.makedirs(prediction_dir, exist_ok=True)
np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_val_prob.npy"
    ),
    val_probabilities,
)
np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_val_labels.npy"
    ),
    val_labels,
)
np.save(
    os.path.join(
        prediction_dir, f"{dataset_name}_exp_" f"{experiment_number}_test_prob.npy"
    ),
    test_probabilities,
)
np.save(
    os.path.join(
        prediction_dir,
        f"{dataset_name}_exp_" f"{experiment_number}_test_labels.npy",
    ),
    test_labels,
)
result = {
    "experiment": experiment_number,
    "learning_rate": config["learning_rate"],
    "batch_size": config["batch_size"],
    "weight_decay": config["weight_decay"],
    "best_validation_f1": best_metric,
    "accuracy": validation_results["eval_accuracy"],
    "precision": validation_results["eval_precision"],
    "recall": validation_results["eval_recall"],
    "f1": validation_results["eval_f1"],
}
print("\nVALIDATION RESULT:")

print(result)
del trainer
del model
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

if os.path.exists(output_dir):

    shutil.rmtree(output_dir)

    print(f"\nDeleted checkpoint folder:" f"\n{output_dir}")
return result
```

Ensemble Code

```
def run_ensemble(dataset_name, bert_experiment, roberta_experiment):

    bert_val_prob = np.load(
        f"{BERT_DIR}/" f"{dataset_name}_exp_" f"{bert_experiment}_val_prob.npy"
    )

    bert_val_labels = np.load(
        f"{BERT_DIR}/" f"{dataset_name}_exp_" f"{bert_experiment}_val_labels.npy"
    )

    roberta_val_prob = np.load(
        f"{ROBERTA_DIR}/" f"{dataset_name}_exp_" f"{roberta_experiment}_val_prob.npy"
    )

   bert_test_prob = np.load(
        f"{BERT_DIR}/" f"{dataset_name}_exp_" f"{bert_experiment}_test_prob.npy"
    )

    bert_test_labels = np.load(
        f"{BERT_DIR}/" f"{dataset_name}_exp_" f"{bert_experiment}_test_labels.npy"
    )

    roberta_test_prob = np.load(
        f"{ROBERTA_DIR}/" f"{dataset_name}_exp_" f"{roberta_experiment}_test_prob.npy"
    )

    X_meta_train = np.concatenate([bert_val_prob, roberta_val_prob], axis=1)

    X_meta_test = np.concatenate([bert_test_prob, roberta_test_prob], axis=1)

    y_meta_train = bert_val_labels

    y_test = bert_test_labels

    meta_learner = LogisticRegression(max_iter=1000, random_state=42)

    meta_learner.fit(X_meta_train, y_meta_train)


    predictions = meta_learner.predict(X_meta_test)


    accuracy = accuracy_score(y_test, predictions)

    precision = precision_score(
        y_test, predictions, average="weighted", zero_division=0
    )

    recall = recall_score(y_test, predictions, average="weighted", zero_division=0)

    f1 = f1_score(y_test, predictions, average="weighted", zero_division=0)

    return {
        "dataset": dataset_name,
        "bert_experiment": bert_experiment,
        "roberta_experiment": roberta_experiment,
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```